CalmGesture
A Gesture to Sound Sensory Regulation Tool for Autism Support Using Deep Learning

Final report on
Deep Learning Project
[ICT-4442]

Submitted By

Sangini Singh 230911020
Tanvi Gupta 230911122
Aditya Nigam 230911366
Shubh Rastogi 230953258



MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

SCHOOL OF COMPUTER ENGINEERING
MANIPAL INSTITUTE OF TECHNOLOGY,
MANIPAL ACADEMY OF HIGHER EDUCATION

OCTOBER 2026

CalmGesture Final Report

Abstract — CalmGesture investigates a local, camera-based interface that maps hand movements to predefined tones. The project compares a multilayer perceptron, a spatial convolutional network implemented in NumPy, a temporal one-dimensional convolutional network, and a gated recurrent unit network. A selected 20BN-Jester subset contains 3,000 clips grouped into rolling, shaking, swiping, tapping, and background. MediaPipe extracts 21 hand landmarks per frame. Motion-preserving normalization and temporal resampling produce 30-by-63 input sequences. A training-only quality filter retains 1,292 clips, while internal validation and held-out testing each retain all 500 clips, including tracking failures. The temporal convolutional model achieves the highest validation macro-F1 of 76.98% and is selected for integration. Its held-out accuracy is 79.80% and macro-F1 is 79.92%. Swiping remains the weakest class, with 55% recall. The selected model powers a browser interface with local inference, stable-prediction gating, four tones, and volume controls. Saved-frame integration checks reveal repeated window resets and stationary-hand false activation. Actual webcam performance and audible output still require validation. These results establish a reproducible gesture-classification prototype, while reliable continuous interaction and any autism-support benefit remain unproven.

Keywords — hand gesture recognition, deep learning, MediaPipe landmarks, temporal convolution, gesture-controlled sound, Jester dataset.

I. Introduction

CalmGesture explores whether ordinary webcam movements can control sound without a physical instrument. The motivating application is an optional sensory interaction interface in an autism-support context. Gesture classification alone cannot establish sensory regulation or therapeutic benefit. The present work therefore evaluates the computational pipeline, not a clinical intervention or diagnostic system.

The scope has evolved since the interim report. The project now uses only an existing Jester subset, with no custom recordings. All four model families have trained on real data, replacing the synthetic pipeline checks previously reported. This report describes the implementation and evidence available on 16 October 2026. Earlier synthetic accuracies are not used as experimental results.

The interaction problem has two distinct stages. First, the system must assign a label to a short sequence of observed hand movement. Second, it must decide whether that label is sufficiently stable to change the sound. A classifier may score well on isolated clips yet behave poorly during transitions, pauses, or missing detections. Consequently, the project evaluates both the offline recognition problem and the behavior of the integrated decision logic. The latter is currently a prototype validation task rather than a completed user evaluation.

A conventional graphical interface provides an explicit button or slider for changing sound. A camera interface instead infers intention from visible movement. This removes the need to touch a control, but introduces ambiguity: a hand may be resting, partly outside the image, or moving for reasons unrelated to the application. The background class therefore has a functional role beyond balancing a dataset. It should prevent undesired activation when no supported musical gesture is intended. The observed static-hand false positive shows why this requirement must be measured separately from overall classification accuracy.

The four foreground categories were chosen for their relevance to the planned interaction, rather than attempting all original Jester categories. Forward and backward rolling share one sound, and left and right swiping share another. This reduces the number of output actions the interface must expose. It also changes the learning task: a directional distinction that matters in the original dataset becomes irrelevant in the grouped labels. All reported scores must therefore be interpreted within this explicit mapping.

The system uses ordinary RGB video as input and compact landmarks as the learned representation. It does not infer emotion, stress, diagnosis, or user preference. The current tone mapping is predefined and has not been validated as calming for any participant. Its practical purpose is to demonstrate a controllable movement-to-audio loop, while preserving an accessible path to mute or stop output. The intended outcome of this project phase is a technically reproducible comparison and an inspectable prototype, with its remaining limitations exposed.

II. Literature Review

Rajagopalan, Dhall, and Goecke introduced SSBD for studying self-stimulatory movements in unconstrained video [1]. Mohammadian Rad et al. studied learned features from accelerometers [2], and subsequently combined convolutional and recurrent models for wearable movement signals [3]. These studies motivate motion modelling but use different sensing tasks and populations. CalmGesture neither trains on SSBD nor infers an autism diagnosis.

Gesture2Music uses a causal temporal convolutional model to convert landmark sequences into musical control events [4]. Its streaming formulation highlights temporal continuity and musical stability. The interim review also included a MediaPipe/CNN sign-language preprint [5]. That paper is withdrawn, so its reported accuracy does not support this report. Gil-Martín et al. evaluate MediaPipe landmark representations with subject-wise validation and find advantages from using the first-frame wrist as a coordinate reference [6]. This supports preserving motion rather than independently centering every frame.

Pooja et al. investigate GRU/LSTM networks for sign-language sequences [7]. Zhang, Liu, and Wu study compact models for mobile action recognition [8]. Linardakis et al. identify occlusion, cross-user generalization, and efficiency as persistent challenges [9]. MediaPipe Hands supplies the practical landmark front end [10]. Together these works motivate our four-

family comparison, but their datasets and protocols differ, so their scores cannot serve as direct baselines for this five-class subset.

The literature differs in three dimensions that affect its transfer to CalmGesture: sensing modality, target vocabulary, and evaluation unit. Accelerometers observe motion directly but require worn devices. Video systems must first detect a hand, so their recognition quality depends on visibility and image conditions. Sign-language systems often distinguish symbolic categories that differ in pose or linguistic motion, while gesture-to-music systems need stable timing and continuous control. Our subset represents coarse interaction categories and therefore does not reproduce any of these tasks exactly.

The wearable studies [2], [3] are useful as evidence that temporal patterns and participant variability deserve attention. They are not evidence that a webcam can recognize the same behavior equally well. Their role in this report is methodological: learned features and sequence models provide alternatives to hand-designed summaries. The experiment must still compare those alternatives on its own input representation and fixed held-out samples. This distinction avoids transferring clinical or sensing conclusions across incompatible datasets.

The study by Gil-Martín et al. [6] is particularly relevant to the preprocessing decision. A coordinate origin fixed at the first wrist position can retain the hand trajectory over a sequence, while independently subtracting each frame's wrist removes much of that trajectory. CalmGesture adopts a related motion-preserving choice for x and y. However, the exact normalization, sequence length, and dataset differ, and our experiment has not ablated this choice. It is a design informed by prior work, not a measured causal explanation for our accuracy.

Gesture2Music [4] also highlights a distinction between recognizing a gesture and producing convincing musical behavior. A stable stream of control events requires decisions about onset, continuation, and silence. CalmGesture currently uses sustained sine tones and a simple confirmation rule, not a learned musical event sequence. The comparison therefore motivates future testing of continuity and false activation rather than a numerical accuracy comparison. The prototype does not claim the same expressive control or latency as that system.

The mobile recognition work [8] motivates reporting computational scope carefully. A short neural forward pass does not include camera acquisition or the time needed to observe an action. Similarly, the survey [9] motivates checking occlusion and cross-user generalization rather than relying on an aggregate score. These considerations become concrete in our results: tracking is weakest for swiping, and the live window can repeatedly reset before any prediction is made. Efficient classification cannot compensate for an unusable input stream.

The resulting research gap for this mini project is a controlled comparison of compact model families on the same selected landmark data, followed by integration of the selected model into a local sound interface. The project does not claim a new state-of-the-art network. Its contribution is the reproducible connection between data preparation, alternative representations, shared evaluation, and the practical failure modes of continuous interaction. Nine items from the interim bibliography remain traceable here, with the withdrawn item explicitly excluded from supporting evidence.

Table I. Comparative Literature Survey

The gaps below describe applicability to CalmGesture; they are our synthesis, not necessarily limitations claimed by the original authors. Results across different datasets are not directly comparable.

Study	Year	Method / data	Main finding / contribution	Research gap / project relevance
Mohammadian Rad et al. [2]	2020 / 2021 revision	CNN; accelerometer signals	Investigates learned features for stereotypical movement detection.	Wearable signals bypass camera tracking; transfer to RGB landmarks remains untested.
Mohammadian Rad et al. [3]	2022	Deep sequence models; wearables	Studies convolutional and recurrent processing of movement signals.	Sensor modality and participant context differ; no direct five-class Jester baseline.
Jeyaraj et al. [4]	2025 / 2026 revision	Gesture2Music; causal temporal model	Maps landmark sequences to musical control events.	CalmGesture still needs stable onset, silence and continuous-stream evaluation.
Verma et al. [5]	2020	MediaPipe and CNN; withdrawn paper	Withdrawn; reported recognition results are excluded from evidence.	Retained only for traceability to the interim review; cannot support a performance claim.
Gil-Martín et al. [6]	2022	Landmarks; subject-wise evaluation	Finds benefits from a first-frame wrist coordinate reference.	Normalization and dataset differ; our motion-preserving choice needs an ablation.
Pooja et al. [7]	2025	GRU / LSTM; sign language	Investigates recurrent modelling of sign sequences.	Symbolic signing differs from coarse sound-control gestures and resting input.
Zhang, Liu and Wu [8]	2023	Deep models for mobile action recognition	Studies compact models for real-time mobile action recognition.	Classifier efficiency alone does not measure camera-to-audio response delay.

Study	Year	Method / data	Main finding / contribution	Research gap / project relevance
Linardakis et al. [9]	2024	Survey of visual gesture recognition	Synthesizes methods and challenges including occlusion and generalization.	A survey does not establish reliability for this subset or continuous interface.
Zhang et al. [10]	2020	MediaPipe Hands; hand tracking	Provides an on-device real-time hand landmark pipeline.	Landmarks are an input representation, not gesture labels; tracking failures remain.
Cho et al. [14]	2024	Gated recurrent encoder-decoder	Learns sequence representations and improves a translation system.	Supports the recurrent design, but not gesture accuracy or optimal sequence length.
Kingma and Ba [15]	2024 preprint	Adam optimizer	Uses adaptive first and second moment estimates for efficient optimization.	Does not determine our best learning rate or resolve class imbalance.
Bai et al. [16]	2022	Convolutional versus recurrent sequence models	Convolutional models outperform canonical recurrent models on varied benchmarks.	Our shallow temporal CNN is not their full TCN; local comparison is still necessary.
Srivastava et al. [17]	2026	Dropout regularization	Randomly drops units during training to reduce co-adaptation and overfitting.	Our dropout rate has not been ablated; observed overfitting is not fully resolved.
Pedregosa et al. [18]	2025	Scikit-learn software	Offers consistent tools for supervised and unsupervised learning.	A software reference does not establish model superiority or prevent split leakage.
Paszke et al. [19]	2025	PyTorch software	Combines imperative model development with efficient execution.	Framework capabilities do not guarantee fair timing or reproducible data preparation.
Lugaresi et al. [20]	2025	MediaPipe perception framework	Supports reusable perception pipelines and device-level performance analysis.	Application-specific tracking continuity, gating and audio behaviour still require tests.

The foundational studies [14]–[17] support gated recurrence, adaptive optimization, temporal convolution and dropout. They motivate design choices rather than numerical performance targets. Scikit-learn [18] supports the MLP baseline, PyTorch [19] supports the temporal CNN and GRU, and MediaPipe [20] supplies the perception framework. The comparison in this report remains specific to the fixed Jester subset.

III. Objectives

The objectives are to compare four classifiers under one fixed data protocol; quantify accuracy, macro-F1, class errors, and classifier latency; and integrate the selected checkpoint into a local gesture-to-tone prototype with explicit testing of tracking loss and false activation.

- The first objective is reproducibility of the four-model comparison. Its acceptance criteria are one declared class mapping, one shared set of clip memberships, training-only feature normalization, and saved predictions or checkpoints sufficient to reproduce the held-out metrics. This objective is supported by the manifest, dataset metadata, saved models, and checkpoint reload checks. It does not require the models to have identical inputs after aggregation, because the representation is part of the architectural comparison.
- The second objective is useful measurement rather than a single headline accuracy. Overall accuracy must be accompanied by macro-F1 and class-level errors, because a model can perform well on shaking and tapping while remaining unreliable for swiping. Classifier timing must state the hardware, batch size, repetition count, and excluded stages. The current experiment meets the offline measurement objective but does not yet establish confidence intervals, statistical significance, or full camera-to-audio responsiveness.
- The third objective is a working local integration path with explicit controls for activation and silence. A session must start only on user action, stop capture when ended, and prevent uncertain predictions from sustaining sound indefinitely. Feature parity between training and live inference is required so that any failure can be investigated without an unnoticed representation mismatch. Automated checks and saved-frame replay support these engineering requirements. A successful live webcam and audible-output evaluation remains a separate acceptance criterion.

These objectives distinguish implemented functionality from validated behavior. The application can execute the extraction, classification, and sound-control path, yet its recognition policy is not reliable enough to claim completion of a user-facing assistive system. In particular, the static-frame replay demonstrates an unmet silence requirement. Treating this as an explicit result helps prioritize the next experiment: measure false activation under resting conditions before expanding the number of gestures or adding richer audio.

The measurable outputs of this phase are thus a five-class dataset manifest, four model artifacts, common validation and held-out results, a class-error analysis, and a local interface. Any future target, such as higher recall or shorter response delay, should be defined before tuning and evaluated on data that did not guide that tuning. Suggested goals are not substituted for measured values in this report.

IV. Methodology

All models use the same extracted landmark sequences and declared train, validation, and test memberships. Model-specific feature aggregation changes the representation but never changes the evaluation samples. Each training procedure restores the checkpoint with minimum validation loss. Across model families, the selection rule is highest validation macro-F1, with validation accuracy as the tie-breaker.

A. Multilayer Perceptron

The baseline computes the temporal mean and standard deviation of each of 63 coordinates, yielding 126 features. A training-fitted StandardScaler precedes hidden layers of 64 and 32 ReLU units and a five-class output. The scikit-learn implementation uses Adam, learning rate 0.001, L2 coefficient 0.0001, batch size 32, and seed 42. A maximum of 500 partial-fit epochs and patience 20 use the shared validation set. The run stops after 26 epochs and restores epoch 6. This baseline tests whether aggregated pose and motion statistics are sufficient without explicit temporal ordering.

For an input coordinate sequence $x(t,j)$, the MLP receives its mean over time and its standard deviation over time. The first statistic describes the average configuration, while the second records the magnitude of variation. Two sequences with the same statistics can nevertheless have different temporal order. The baseline therefore represents an intentionally compact hypothesis: many grouped gestures may be separable from posture and motion amplitude without learning an explicit sequence.

The scaler estimates one mean and standard deviation per aggregate feature using training clips only. Validation and test features use those saved values. Fitting the scaler again on a test batch would change the representation and leak information from evaluation samples into preprocessing. Saving the fitted scaler with the classifier also makes inference independent of the other samples present in a request.

Each hidden layer applies an affine transformation followed by ReLU. The final classifier produces class probabilities through a multiclass output. Training repeatedly performs one partial-fit epoch and evaluates the separate shared validation set. A copy of the best model is retained whenever validation loss improves. This avoids using scikit-learn's internal early-stopping split, which would create a different validation protocol from the other models.

The MLP offers a low-cost reference for judging whether added temporal machinery is worthwhile. If a much more complex sequence model delivered no benefit, the aggregated representation would remain attractive for deployment. In this experiment, its 76.6% held-out accuracy is competitive but below the selected temporal CNN. This comparison establishes a practical baseline, although a single run is insufficient to quantify how stable the difference would be under another initialization.

The main representational limitation is that mean and standard deviation do not encode motion direction or timing. A leftward sweep and its reversed sequence can yield similar aggregate features. Grouping left and right swipes makes some directional ambiguity acceptable here, but the same aggregation can also obscure the distinction between a repeated movement and a short isolated excursion. These are reasons to compare full-sequence models, not explanations proven by the observed confusion matrix.

B. Spatial Convolutional Neural Network

The NumPy baseline forms a two-channel 21-by-3 pose image: temporal means in one channel and standard deviations in the other. Training-fitted channel normalization precedes two padded 3-by-3 convolutions with 8 and 16 channels, ReLU activations, flattening, a 32-unit dense ReLU layer, and five logits. Training uses cross-entropy with gradient-descent updates, learning rate 0.05, batch size 16, seed 0, a 60-epoch ceiling, and patience 10. It restores epoch 4 after 14 epochs. Numerical gradient tests check the custom implementation. This model tests local patterns in aggregated landmarks. The earlier proposed framework port remains unimplemented; the reported CNN is explicitly the NumPy baseline.

The pose image is a structured numerical array rather than an RGB image. Its height indexes hand landmarks and its width indexes x, y, and z. A spatial convolution shares weights across neighboring array entries. This is computationally convenient, but the imposed neighborhood is only an approximation to hand anatomy: successive landmark indices do not always represent equally meaningful spatial neighbors, and coordinate axes are not interchangeable image pixels.

The first convolution transforms two input channels into eight feature maps. The second transforms those maps into sixteen. Padding preserves the 21-by-3 spatial extent, after which flattening produces 1,008 values for the dense classifier. ReLU introduces nonlinearity, allowing combinations of pose and variability to influence the output. This architecture is deliberately small enough to implement and inspect directly in NumPy.

The loss is multiclass cross-entropy. Gradients propagate from the logits through the dense and convolutional layers, and parameter updates use a fixed learning rate. An earlier gradient scaling problem was corrected so that the batch-mean factor is applied once rather than twice. Finite-difference checks compare analytic derivatives with numerical estimates on small inputs. These checks are important for a custom network because a training curve alone cannot prove that backpropagation is correct.

Checkpoint selection uses the minimum validation loss, and the saved artifact includes the weights and preprocessing state needed for prediction. The run's validation macro-F1 is 75.97%, slightly above the MLP's 75.20%, but held-out accuracy

is lower at 74.0%. This reversal cautions against interpreting small validation differences as guaranteed test improvements. It also emphasizes why the held-out set must remain outside the selection process.

The temporal mean and standard-deviation channels retain more information than a mean-only pose image, but they still discard ordering. The earlier synthetic comparison that motivated this representation is not a real-data ablation. It cannot establish the size of the benefit on Jester. A proper ablation would hold initialization policy, split, optimizer, and model capacity as constant as possible while changing only the input channels. That experiment has not been performed.

The report retains this model as the from-scratch spatial baseline. The interim plan mentioned a framework port and architecture adaptation, but those changes are not part of the measured implementation. Reporting the actual NumPy network avoids attributing results to an unimplemented model and preserves a clear link between the methodology and the saved checkpoint.

C. One Dimensional Convolutional Neural Network

The temporal CNN processes the full 30-by-63 sequence. Convolutions operate over time with 64 channels, kernel sizes 5 and 3, same-length padding, and ReLU. Global temporal average pooling, dropout 0.2, and a linear five-class head produce logits. The model contains 32,901 trainable parameters. Per-coordinate means and standard deviations are fitted on training frames only. Cross-entropy and Adam use learning rate 0.001, batch size 32, zero weight decay, seed 42, and a 50-epoch ceiling. Patience 10 stops training at epoch 23 and restores epoch 13. Local temporal filters can capture short movement patterns that aggregation discards.

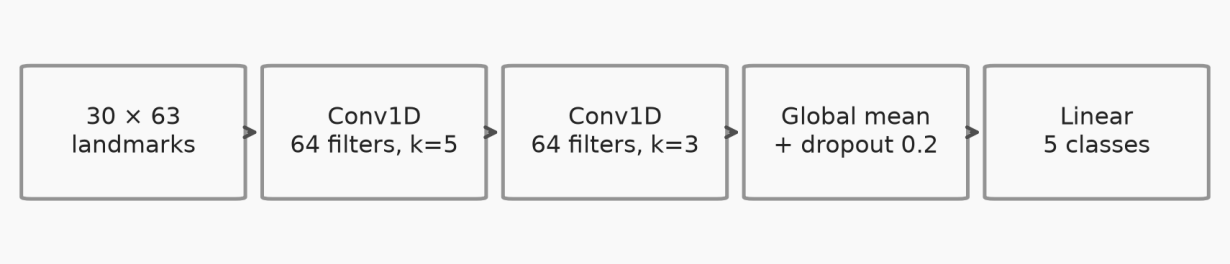


Fig. 1. Temporal CNN architecture. ReLU follows each convolution.

The input is transposed inside the network so that the 63 landmark coordinates act as channels and the 30 samples form the temporal axis. A convolutional output at a given time combines neighboring observations across all input coordinates. The first kernel spans five resampled frames, and the second spans three feature positions. With unit stride and no dilation, the stacked local receptive field covers seven frames before temporal pooling.

Global average pooling reduces each of the 64 temporal feature maps to one value. This keeps the output dimension independent of the temporal positions within the fixed sequence and reduces the number of dense parameters. It also loses information about the absolute location of a detected pattern in the clip. That trade-off is reasonable for grouped gestures whose occurrence matters more than their exact onset, but may become a limitation for continuous sound timing.

Dropout randomly removes a fraction of pooled features during training. It is disabled during validation and inference. Consequently, training loss is measured under a different stochastic condition from validation loss, in addition to the difference in samples. The observed widening gap still indicates poor generalization late in training, but the losses should not be interpreted as identical measurements on two interchangeable datasets.

The first convolution has 20,224 parameters, the second has 12,352, and the final linear layer has 325. Their sum is 32,901. This parameter accounting follows the saved architecture rather than a generic description of a temporal CNN. The model's small size helps make a CPU deployment feasible, although landmark extraction and the observation window dominate other parts of the interaction pipeline.

The sequence-level model is not causal in the sense of making a prediction from one arriving frame alone. It classifies a collected window, and its padded temporal filters can access both earlier and later positions within that window. The application can update this sliding window repeatedly, but it still needs enough observations to fill it. This distinguishes the implementation from streaming event models designed specifically for low-delay musical control.

D. Gated Recurrent Unit Network

The GRU receives the same normalized 30-by-63 sequence. A single recurrent layer with hidden size 64 summarizes the sequence through its final hidden state. Dropout 0.2 and a linear five-class layer produce the output. It uses the same optimizer, learning rate, batch size, seed, epoch ceiling, and patience as the temporal CNN. Training stops after 14 epochs and restores epoch 4. This provides a recurrent alternative for temporal modelling without assuming that it will outperform convolution.

A GRU maintains a hidden state that changes as each observation arrives. Its reset and update gates control how much previous state and new evidence contribute to the next state. This gives the network a mechanism for modelling order and duration without explicitly flattening every time step into a very large vector. In CalmGesture, the final hidden state represents the entire resampled clip and feeds the classifier.

The single-layer model has an input dimension of 63 and hidden dimension of 64. Each of its three gated transformations has input weights, recurrent weights, and biases. The recurrent layer therefore contains 24,768 parameters, and the five-

class linear head adds 325, for a total of 25,093. This is fewer parameters than the temporal CNN, but sequential recurrence can still make a forward pass slower on the measured CPU.

The GRU uses the same sequence normalization as the temporal CNN. Its optimization schedule also uses Adam and the same nominal learning rate, batch size, seed, and early-stopping patience. This reduces some procedural differences, but does not make the model families perfectly matched: their optimization landscapes, inductive assumptions, and effective capacities differ. Equal hyperparameters are a controlled starting point, not proof that each model received its best possible tuning.

The restored epoch is early in training, at epoch 4. The resulting validation macro-F1 is 72.67% and held-out accuracy is 76.0%. The lower validation score does not show that recurrence is generally unsuitable for gestures. It shows that this particular GRU and training configuration did not outperform the alternatives on this representation and subset. More extensive hyperparameter searches would require a clearly bounded validation protocol.

Using only the final hidden state also means that information from early frames must survive the recurrent updates to influence the decision. Pooling over all hidden states or adding attention would change this behavior, but neither is implemented. Such changes belong to future experiments rather than the description of the current network. The present model provides a simple, reproducible recurrent comparison.

Together the four architectures separate two design choices: using aggregated statistics versus the full temporal sequence, and applying dense, convolutional, or recurrent transformations. This structure supports discussion of plausible mechanisms, but it is not a factorial ablation. The comparison changes more than one factor at a time. The reported winner is therefore the best tested configuration under the stated validation rule, not a universal ranking of neural network families.

V. Experimental Setup

A. Dataset and Shared Splits

The selected 20BN-Jester subset [11] contains 3,000 clips and 107,026 JPEG frames. Eight source labels map to five project classes: Rolling Hand Forward/Backward, Shaking Hand, Drumming Fingers, Swiping Left/Right, and No gesture/Doing other things. There are initially 600 candidates per grouped class. Official training data supplies 2,000 training candidates and 500 internal-validation clips. Official validation supplies the 500 held-out test clips. Thus the candidate train/validation/test ratio is 66.67/16.67/16.67%. This is not the official full Jester test benchmark.

TABLE II. DATASET MEMBERSHIP AFTER TRAINING QUALITY FILTERING

Class	Train	Validation	Test
Background	400	100	100
Rolling	188	100	100
Shaking	308	100	100
Swiping	96	100	100
Tapping	300	100	100
Total	1292	500	500

Only foreground training clips with at least 50% successful hand detection are retained. All background training clips and every validation/test clip remain. The retained ratio is 56.37/21.82/21.82% of 2,292 clips. Filtering removes 708 training candidates and creates substantial class imbalance. No augmentation, oversampling, SMOTE, or class weighting is applied. Subject-disjointness has not been established for this selected protocol.

The subset construction controls the number of candidates per grouped class before extraction. It does not guarantee equal numbers of usable tracked sequences. Once training foreground clips are filtered, swiping has approximately one quarter of its candidate count remaining, while shaking and tapping retain much larger fractions. This creates a useful distinction between a balanced annotation manifest and an imbalanced learning matrix. Both counts are reported so that the filter cannot hide the data actually seen by the models.

Retaining every evaluation clip is important for the intended pipeline. If a difficult test clip were discarded after tracking failure, the reported score would describe only successful extraction cases. Here, zero or weak landmark features still reach the classifier, exposing the consequences of the front end. However, filtering training while retaining all evaluation clips also creates a distribution mismatch. The model sees fewer low-quality foreground examples during training than it encounters during evaluation.

The official split names and the project split names have different meanings. Internal validation supports checkpoint and model selection. The project's held-out test is sampled from official validation annotations, not from the original benchmark test protocol. Reporting the source membership prevents confusion with published full-dataset leaderboard results. No custom recordings or SSBD clips appear in any of the three project splits.

The available experiment metadata records clip identifiers and split assignments, but this report does not claim participant independence. Without a verified subject mapping, a clip-level split cannot establish performance on entirely unseen users. This limitation is particularly relevant for an interface intended to work across different hands, movement styles, and

camera conditions. Future evaluation should either obtain valid participant information or clearly retain the narrower claim of clip-level generalization.

B. Landmark Preprocessing

MediaPipe Tasks HandLandmarker 0.10.32 processes one hand at a nominal source rate of 12 frames per second. The pipeline aspect-corrects x and z, subtracts the first valid wrist position from x/y to preserve travel, subtracts each frame's wrist depth from z, and divides by the median valid palm scale. Missing detections reuse the previous valid landmarks, with initial gaps filled from the first valid frame. Sequences with no valid detection become zero features. Linear resampling produces 30 frames, each with 21 XYZ landmarks. Model normalization is fitted exclusively on training data.

Aspect correction matters because normalized image x and y coordinates use different pixel dimensions when the frame is not square. Multiplying x and z by the width-to-height ratio expresses them relative to the height scale used for y. The first valid wrist becomes the horizontal and vertical origin for the whole clip. Unlike per-frame wrist centering, this preserves how the hand travels after the initial position.

The palm scale normalizes for approximate hand size and camera distance. Using a median across valid frames reduces the influence of a single unusual scale estimate. It does not make the representation completely invariant to perspective, rotation, or articulation. Depth values from the landmark estimator are also not equivalent to calibrated metric measurements, so the model should be understood as using relative coordinates.

Filling missing frames keeps the array shape continuous, but introduces repeated observations. A short gap can preserve a reasonable trajectory approximation, while a long gap may make a moving gesture look static. Linear resampling then maps clips with different lengths to 30 positions. This preserves a coarse ordering while changing the relationship between sample index and physical time. Fast and slow performances can become more similar, but detailed timing information is reduced.

Zero-filled examples are retained when no hand is detected. Their numerical values do not directly mean a correctly observed resting hand. They mean that the extraction pipeline supplied no valid landmarks. A model may learn to associate such arrays with background, which can suppress some failures but can also misclassify foreground gestures with poor tracking. The swiping confusion pattern makes this distinction important to investigate.

The live implementation calls the same feature routine as offline preprocessing. The parity test compares features produced from the same landmark window, preventing a silent difference in normalization or resampling between training and deployment. It does not verify that live camera frames have the same distribution as Jester clips. Camera angle, background, hand distance, and movement duration can all differ even when the mathematics is identical.

C. Metrics and Reproducibility

Accuracy is the proportion of correctly classified clips. For each class, precision equals TP/(TP+FP), recall equals TP/(TP+FN), and F1 is the harmonic mean of precision and recall. Macro-F1 averages class F1 values equally, making weak classes visible despite training imbalance. Confusion matrices use actual classes as rows and predictions as columns. No threshold tuning uses the held-out test set.

Training ran on an Apple M1 CPU. Temporal models use PyTorch 2.14.1 and four threads. Saved checkpoints reproduce all four reported held-out results. Dataset and preprocessing hashes accompany the metrics. The project repository [12] contains the pipeline and tests, while raw data and trained model files remain outside normal Git commits. Per-clip classifier latency uses batch size one, 10 warm-up calls, and 100 timed repeats. It excludes camera capture, landmark extraction, the movement window, and sound playback.

Checkpoint selection and model-family selection use different criteria. Within each training run, minimum validation cross-entropy selects the saved epoch. Across those saved candidates, validation macro-F1 chooses the family for integration. Cross-entropy evaluates the probability assigned to the correct class, whereas macro-F1 evaluates discrete decisions across classes. These criteria need not reach their optimum at the same epoch, so the report explicitly states both rather than implying one uniform selection statistic.

The fixed held-out test contains 100 examples per class. This makes class recall directly readable as the number correctly identified out of 100. Macro-F1 still adds information beyond overall accuracy because precision depends on false positives from other categories. Background can have high recall while also attracting many incorrectly rejected foreground clips. The confusion matrix is therefore necessary to interpret the balance between activation and silence.

Latency measurements are repeated forward-path timings, not a throughput benchmark or a user response-time study. A median reduces the influence of unusually slow calls, and the saved p95 provides a view of the slower tail. Differences near a few thousandths of a millisecond should not be treated as practically meaningful without repeated timing sessions. The larger system includes image decoding, landmark detection, scheduling, transport, and audio rendering, none of which appears in the classifier-only timing.

D. Local Application

The browser captures camera frames only after Start. A loopback Python service extracts landmarks and applies the training feature function to a 36-frame rolling window at a target 12 fps. Three consecutive predictions with score at least 0.75 enable a tone. Background, low confidence, or missing hands silence output. Rolling, shaking, tapping, and swiping map to C4, E4, G4, and A4 sine tones. Mute, volume, and Stop controls are available. Frames are processed locally and are not saved. Gesture Synth [13] inspired the interaction concept; the project uses its own trained classifier and implementation.

The stability gate counts consecutive confident labels. A changed candidate, a low-confidence result, or background clears the active state. Missing-hand frames silence output immediately, and sustained tracking loss clears the accumulated window. A timestamp gap or a detected hand change also resets state. These choices prioritize avoiding a stale tone, but can make the interface difficult to activate when tracking is intermittent.

The sound layer uses oscillator gain ramps to reduce abrupt starts and stops. Its volume control changes output gain, and mute overrides gesture activation. When the tab becomes hidden or the session ends, capture stops and the application requests server cleanup. These are control-path behaviors implemented in the browser code. Their presence does not replace an audible test on the presentation machine.

The confidence threshold of 0.75 and three-confirmation rule are provisional settings. They were not calibrated against a labeled continuous validation stream. High softmax confidence can occur on an unfamiliar or static input, as the replay test demonstrates. A more reliable activation policy may need explicit motion evidence, class-dependent thresholds, or a better-defined background vocabulary rather than simply raising one global score threshold.

A practical demonstration should begin with the server and model files available locally, then verify camera permission, hand tracking, and audio at a comfortable volume. It should include resting behavior as well as supported gestures. A failure to activate is observable and should be reported rather than concealed by repeatedly choosing only favorable movements. No camera recording is necessary for this operational check, consistent with the current dataset-only development scope.

VI. Results and Discussion

A. Common Model Comparison

TABLE III. REAL JESTER SUBSET RESULTS AND CPU CLASSIFIER LATENCY

Model	Val F1 (%)	Test acc (%)	Test F1 (%)	Median ms
MLP	92.20	93.60	93.80	0.114
NumPy CNN	92.97	91.00	91.60	0.106
1D-CNN	93.98	96.80	96.92	0.111
GRU	89.67	93.00	92.84	0.243

The temporal CNN leads validation macro-F1 and is selected before test results are used for reporting. It correctly classifies 399 of 500 held-out clips and improves accuracy over the MLP baseline by 3.2 percentage points. Its measured classifier latency is similar to the aggregate-feature baselines, while the GRU is slower in this CPU setup. The saved temporal CNN training timer is 8.04 seconds, excluding data preparation.

Preserving order is a plausible explanation for the temporal CNN's advantage, because mean/std representations discard the arrangement of movements. However, the comparison also changes architecture and optimization, so it cannot isolate that cause. The GRU's lower score shows that recurrent modelling alone does not guarantee improvement. Results come from one initialization per model, with the NumPy CNN using a different seed. No confidence intervals, repeated-seed study, controlled ablation, or directly comparable state-of-the-art experiment is available.

The test results are 383 correct clips for the MLP, 370 for the spatial CNN, 399 for the temporal CNN, and 380 for the GRU. These counts help interpret percentage differences on a finite sample. The 16 additional correct predictions made by the temporal CNN relative to the MLP do not automatically establish a statistically significant improvement. A paired analysis would need the per-clip pattern of disagreements, and a broader study would need repeated runs.

Validation and test performance are reasonably close for the selected model: 77.0% validation accuracy and 79.8% held-out accuracy. The higher test score does not indicate that the model learned from testing. It can arise from sample composition and difficulty. Conversely, the spatial CNN's lower test score illustrates that selection uncertainty remains even when all models share the same split protocol.

The saved classifier p95 timings are approximately 0.119 ms for the MLP, 0.126 ms for the spatial CNN, 0.120 ms for the temporal CNN, and 0.250 ms for the GRU. These are all small relative to the nominal movement collection window. Optimizing classifier execution alone is therefore unlikely to make the current interaction feel immediate. A useful optimization should identify which stage dominates observed response delay before changing the architecture.

The main model choice combines validation performance with a practical integration path. The temporal CNN has the strongest measured validation macro-F1, a small parameter count, and a saved normalization/checkpoint format. Its superiority is limited to the tested configurations. The study does not exhaust learning rates, hidden dimensions, kernels, recurrent pooling strategies, or alternative feature sets. Additional tuning would have to preserve the held-out test as a final assessment rather than repeatedly using its errors to choose settings.

B. Training Dynamics

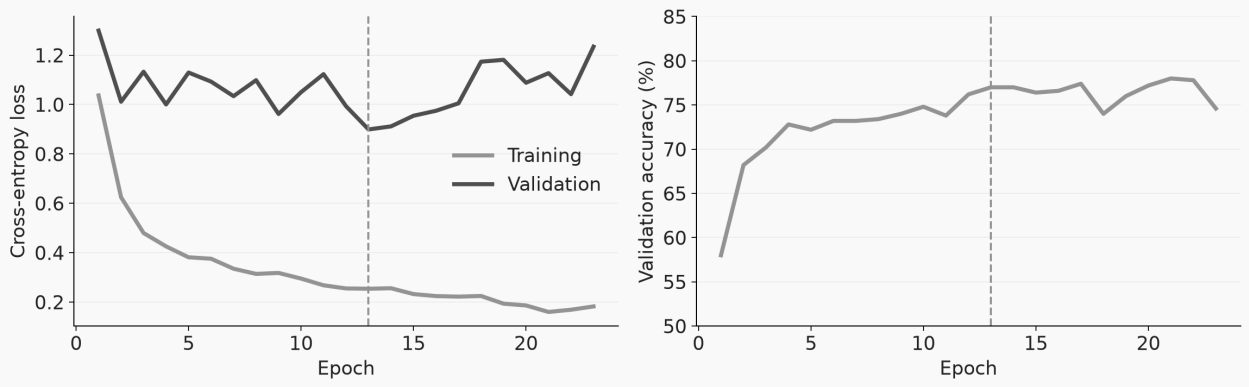


Fig. 2. Temporal CNN loss and validation accuracy. Dashed lines mark restored epoch 13.

Training loss falls from 1.037 to 0.254 by epoch 13, where validation loss reaches its minimum of 0.899. Later training continues to reduce training loss while validation loss generally increases, reaching 1.234 at epoch 23. This gap is evidence of overfitting. Restoring epoch 13 limits the degradation. Validation accuracy at the selected checkpoint is 77.0%. Training accuracy was not logged, so Fig. 2 deliberately shows only the recorded validation accuracy curve.

The loss history shows why the final training epoch should not automatically be used for deployment. At epoch 23 the model has continued fitting the training distribution but assigns worse probabilities to validation examples than at epoch 13. Restoring the minimum-loss weights is therefore an actual intervention in the experiment, not merely a recommendation. The selected checkpoint can be reloaded independently to reproduce the reported score.

Cross-entropy is sensitive to confident mistakes. A model can achieve similar validation accuracy across two epochs while assigning lower probability to the correct labels on the examples it misses. This explains why the loss-based best epoch need not coincide with the highest recorded validation accuracy. The experiment uses the rule declared before evaluation instead of choosing whichever epoch gives the most favorable test result.

Potential explanations for the generalization gap include limited training size, class imbalance after quality filtering, and different detection quality in training and evaluation. The current curves cannot identify which factor dominates. Dropout and early stopping are already present, but stronger regularization is not automatically the correct fix: it might also reduce useful capacity. Controlled changes should compare validation behavior and class recall together.

No synthetic or reconstructed training-accuracy curve is included. The available training history records loss, while the temporal model's validation history also records accuracy. Preserving that distinction is important because a plausible-looking curve would be visually convincing but unsupported. Future training instrumentation can record both accuracies in evaluation mode without changing the existing experiment's historical evidence.

C. Class Errors

Class recalls are 83% for background, 79% for rolling, 92% for shaking, 55% for swiping, and 90% for tapping. Of 100 swiping test clips, 28 are predicted as background. Only 96 of 400 swiping training candidates pass the quality filter, and 81 of 100 swiping test clips have detection rates below 50%. This association suggests a tracking bottleneck, but a controlled experiment is needed to separate tracking effects from representation and training imbalance.

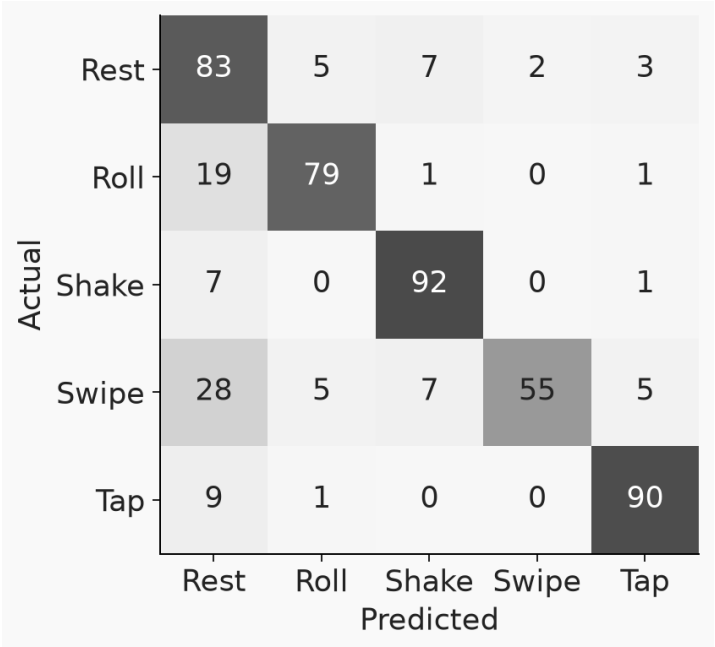


Fig. 3. Selected model confusion matrix on 500 held-out clips. Each class has 100 clips.

The background column receives 83 true background clips and 63 foreground clips: 19 rolling, 7 shaking, 28 swiping, and 9 tapping. Its recall is therefore 83%, but its precision is only 83/146, or about 56.85%. This means that a background decision is often an incorrect rejection of a real gesture in the held-out set. In an interactive system, those errors would appear as silence or missed activation rather than an obviously wrong musical note.

By contrast, the swiping column contains 55 correct swipes and only two background clips. Its precision is about 96.49%, while recall is 55%. The model is therefore conservative about assigning this class under the isolated-clip protocol. This asymmetry is lost in a single accuracy number. It suggests studying how to recover missed swipes while monitoring whether any change increases false activation.

The remaining class precisions are approximately 87.78% for rolling, 85.98% for shaking, and 90.00% for tapping. Their F1 values are approximately 83.16%, 88.89%, and 90.00%, respectively. Background F1 is 67.48% and swiping F1 is 70.06%. The average is the reported macro-F1 of 79.92%. These values are derived directly from the saved confusion matrix, not from an additional evaluation run.

D. Illustrative Predictions and Sensitivity

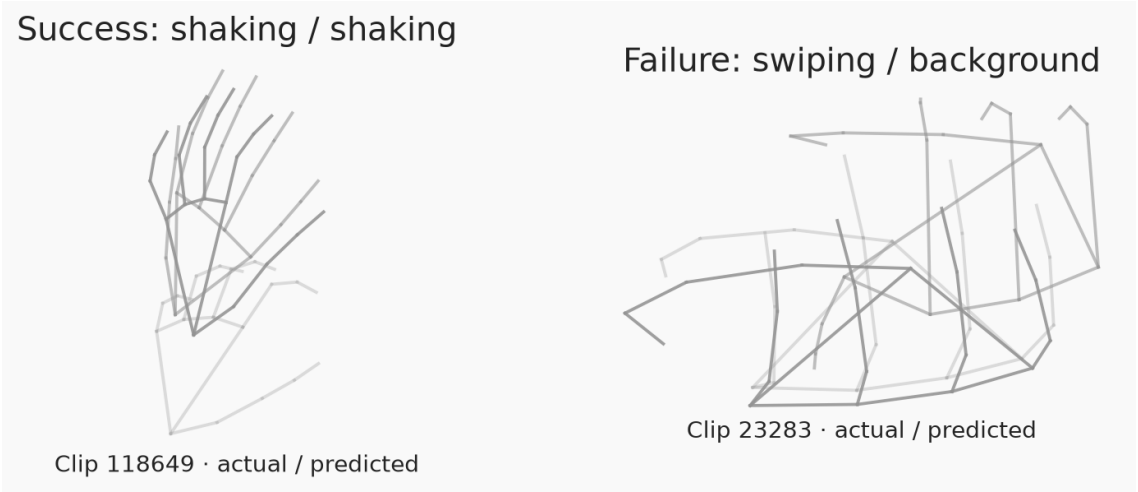


Fig. 4. Normalized hand skeletons at three sampled frames. Labels show actual and predicted classes.

Held-out clip 118649 is correctly recognized as shaking, with a model score of 0.9986 and detection rate 91.9%. Clip 23283 is a swiping example predicted as background, with score 0.5628 and detection rate 61.1%. These examples illustrate success and failure rather than estimating frequency. The scores are uncalibrated softmax outputs, not probabilities of correctness.

The accompanying reproducible visualization script computes absolute gradients of the predicted logit with respect to standardized input coordinates, sums XYZ contributions, and normalizes within the example. Such sensitivity maps can highlight influential input locations but do not prove the model's reasoning. Landmark extraction errors and sequence normalization can affect them, so no causal or clinical interpretation is assigned.

The selected success example has high tracking coverage and a clear correct label. The selected failure retains more than half of its frames, yet the classifier chooses background. This shows that exceeding the quality threshold does not guarantee correct recognition. The threshold is a coarse inclusion rule, not a measure of whether the observed landmarks faithfully encode the intended gesture.

The skeleton plots visualize normalized model inputs at three sampled positions in time. They do not show the original RGB image, the full continuous trajectory, or confidence values from the landmark estimator. Apparent crossings can result from perspective, normalization, and overlaying multiple frames. A qualitative conclusion should therefore combine the figure with the clip's identity, tracking rate, and prediction rather than treating the plot as a complete diagnosis of the error.

Input-gradient sensitivity measures how the selected logit would change under small changes to the standardized coordinates at the current example. Large gradients identify local sensitivity, not necessarily the most semantically important joint. Correlated coordinates, saturating nonlinearities, and the normalization scale can affect the visualization. Repeating the analysis across multiple examples and checking perturbations would provide stronger evidence than one heatmap.

This qualitative analysis is illustrative rather than a new benchmark. The examples were selected after examining predictions, so they cannot estimate the frequency of successes or failures. Frequency comes from the complete confusion matrix. The examples instead make the numerical result inspectable and motivate specific hypotheses for a future controlled study.

E. Continuous Interaction Limitations

Automated checks pass for feature parity, prediction gating, tracking loss, hand changes, and timestamp handling. Saved-frame replays of clips 340 and 887 repeatedly reset the movement window before classification. Repeating a single visible-hand frame 48 times reaches inference but falsely activates shaking. These checks establish integration behavior and reveal defects; they do not demonstrate reliable live gesture recognition. Actual webcam accuracy, browser audio playback, full-

loop latency, and user acceptability remain unvalidated. The nominal three-second collection window also prevents interpreting sub-millisecond classifier timing as immediate sound response.

The static replay is especially informative because it demonstrates a failure that isolated-clip accuracy does not reveal. A continuously visible hand with no actual movement can still produce a high-confidence foreground label. The model was trained to classify extracted clips, not explicitly to detect an onset of movement in an unlimited stream. The application currently relies on background classification and confidence gating to bridge this difference, which is insufficient in the observed case.

Repeated resets create a different failure mode. Even when most incoming frames contain a hand, a short run of missing detections or a change in handedness output can erase the accumulated context. The next prediction then waits for another complete window. A user may experience this as unpredictable delay rather than an incorrect class label. The relevant metric is not just accuracy on windows that reach inference, but also the proportion of attempted interactions that produce a decision within an acceptable time.

These failures support a staged validation plan. First, replay known data through extraction and classification to check state transitions. Second, test stationary and transition segments for unintended activation. Third, assess a supervised webcam session and audible output. Finally, measure repeatability across conditions. Each stage needs a defined denominator, such as attempted gestures, minutes of resting input, or total transitions. At present, only the first integration checks and limited failure probes have been completed.

VII. Conclusion

- CalmGesture now has a complete real-data classification pipeline, four trained and reloadable model families, and a local gesture-to-sound prototype. The temporal CNN is the strongest model under the common validation rule, achieving 79.80% held-out accuracy and 79.92% macro-F1. Its compact temporal architecture provides a useful integration candidate, but weak swiping recognition and false activation during static replay limit practical use. The work supports further engineering evaluation, not a claim of therapeutic benefit or deployment readiness.
- The strongest conclusion is that the complete offline pipeline can produce and reproduce meaningful real-data results. All models use common declared samples and retain difficult evaluation clips. This is substantially more informative than the earlier synthetic checks, which established execution but not real-world recognition quality. The move from synthetic sequences to Jester also exposes tracking as a major source of error that a classifier-only experiment would overlook.
- The model comparison supports choosing the temporal CNN for the next integration iteration, rather than declaring the architecture universally best. Its advantage over the MLP is modest in absolute count, and the study contains only one run per configuration. The evidence is sufficient for a project engineering choice, but stronger comparative claims would require repeated experiments and uncertainty estimates.
- The present state therefore separates three outcomes: real-data classification is implemented and measured; the camera-to-sound path is implemented but incompletely validated; and usefulness for autism support is untested. Keeping these outcomes separate makes the report's claims consistent with the actual artifacts. It also provides a practical basis for deciding what must improve before a final demonstration or a future participant study.

VIII. Future Work

The immediate priority is to prevent sound from stationary or uncertain inputs. Candidate changes include explicit motion thresholds, better background examples from the existing dataset, and validation-based tuning of the stability gate. Tracking tolerance should handle short interruptions without mixing unrelated movements. Improving swiping requires studying landmark coverage and whether training-only balancing or revised filtering improves recall without increasing false triggers.

Controlled ablations should remove retained wrist motion, vary the observation window, and compare class balancing while keeping the same splits. Repeated seeds and confidence intervals would quantify uncertainty. A shorter or incremental window could reduce response delay, but must be tested against recognition quality. Camera-to-audio timing should include capture, transport, landmark extraction, feature computation, decision gating, and playback rather than classifier time alone.

The current plan continues to use the existing Jester dataset, with no custom recordings. A supervised live demonstration can test operation without retaining video. Any future study involving autistic users would require an appropriately designed evaluation, participant consent, and domain expertise before claiming usefulness. The proposed spatial CNN framework port is still a scope item to resolve with the team; it must not be described as completed.

A useful first ablation would compare the existing motion-preserving normalization with per-frame wrist centering. The split, model, optimizer, and seed policy should remain fixed, and the change should be evaluated on validation macro-F1 and per-class recall. Because swiping depends strongly on translation, its recall is a relevant outcome, but the experiment must also check whether other classes become less stable. The current report does not assign a numerical gain to this proposed change.

A second study should vary quality filtering and class balance independently. One comparison could retain more low-detection training clips while keeping evaluation unchanged. Another could use training-only weighted loss or sampling. These changes address different mechanisms: the first changes the quality distribution, while the second changes how

often classes influence optimization. Combining them immediately would make it harder to explain an improvement or regression.

A third study should test observation-window duration. Short windows may reduce response delay but omit parts of a rolling or tapping cycle. Long windows contain more context but can blend successive gestures. Evaluation should include transitions and resting periods, not only presegmented clips. The decision policy should be selected on a dedicated validation stream and assessed once on held-out streams.

Operational evaluation should report at least correct activations, missed activations, wrong-tone activations, false triggers during rest, and time from movement onset to sound. These outcomes reflect what a user experiences. Audio gain ramps, mute behavior, and session cleanup should also be checked on the actual presentation machine. None of these engineering tests requires claiming clinical efficacy.

Before submission, the team should review the implementation details associated with each model and keep the report, presentation and application documentation consistent. The remaining framework-port decision and live validation status should be updated when further experiments are completed.

ACKNOWLEDGEMENT

We express our gratitude to the School of Computer Engineering, Manipal Institute of Technology, Manipal Academy of Higher Education, for the academic setting in which this ICT 4442 mini project was undertaken. The project provided an opportunity to connect deep learning concepts with data preparation, model comparison, reproducibility and the challenges of a working interactive application.

We acknowledge the creators and providers of the Jester dataset for making gesture data available for research. We also thank the authors of the studies reviewed in this report for the methodological foundations and evaluation perspectives that informed our work. The gesture-synth repository provided inspiration for exploring a connection between hand movement and sound.

We are grateful to the open-source communities behind MediaPipe, NumPy, scikit-learn, PyTorch and the scientific visualization tools used in the project. Their software enabled landmark extraction, comparative experiments and the presentation of results. We acknowledge the collaborative effort of our team across the four model assignments, experimental interpretation and preparation of the project documentation.

REFERENCES

[1] S. S. Rajagopalan, A. Dhall, and R. Goecke, "Self-Stimulatory Behaviours in the Wild for Autism Diagnosis," in Proc. IEEE ICCV Workshops, 2013. Available: https://rolandgoecke.net/research/datasets/ssbd/

[2] N. Mohammadian Rad et al., "Convolutional Neural Network for Stereotypical Motor Movement Detection in Autism," arXiv:1511.01865, 2015, revised 2016. https://arxiv.org/abs/1511.01865

[3] N. Mohammadian Rad et al., "Deep Learning for Automatic Stereotypical Motor Movement Detection using Wearable Sensors in Autism Spectrum Disorders," arXiv:1709.05956, 2017. https://arxiv.org/abs/1709.05956

[4] R. Jeyaraj, B. Subramanian, K. Gangadharan, and A. Paul, "Gesture2Music: A Low-Latency Real-Time Framework for Continuous Gesture-Driven Music Generation," arXiv:2511.00793, 2025, revised 2026. https://arxiv.org/abs/2511.00793

[5] A. R. Verma et al., "Enhancing Sign Language Detection through Mediapipe and Convolutional Neural Networks (CNN)," arXiv:2406.03729, 2024, WITHDRAWN. Cited only to correct the interim review. https://arxiv.org/abs/2406.03729

[6] M. Gil-Martín, M. R. Marini, I. Martín-Fernández, S. Esteban-Romero, and L. Cinque, "Hand Gesture Recognition Using MediaPipe Landmarks and Deep Learning Networks," in Proc. ICAART, vol. 3, pp. 24–30, 2025, doi: 10.5220/0013053500003890.

[7] S. Pooja, Prem Sai N, Aravinth A, Prithivirai R, and Manikandan P, "American Sign Language Recognition Using GRU and LSTM," in Proc. INCOFT, vol. 3, pp. 659–666, 2025, doi: 10.5220/0013639700004664.

[8] C.-L. Zhang, X.-X. Liu, and J. Wu, "Towards Real-Time Action Recognition on Mobile Devices Using Deep Models," arXiv:1906.07052, 2019. https://arxiv.org/abs/1906.07052

[9] M. Linardakis, I. Varlamis, and G. Th. Papadopoulos, "Survey on Hand Gesture Recognition from Visual Input," arXiv:2501.11992, 2025. https://arxiv.org/abs/2501.11992

[10] F. Zhang et al., "MediaPipe Hands: On-device Real-time Hand Tracking," arXiv:2006.10214, 2020. https://arxiv.org/abs/2006.10214

[11] Qualcomm, "Jester Dataset," online dataset. Accessed Oct. 16, 2026. https://www.qualcomm.com/developer/software/jester-dataset

[12] CalmGesture project, source code and experiment artifacts, branch codex/sangini, 2026. https://github.com/aditya1nigam8/calmgesture/tree/codex/sangini

[13] ericwei97-cloud, "gesture-synth," GitHub repository, accessed Oct. 16, 2026. https://github.com/ericwei97-cloud/gesture-synth

[14] K. Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," arXiv:1406.1078, 2014. https://arxiv.org/abs/1406.1078

[15] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," arXiv:1412.6980, 2014. https://arxiv.org/abs/1412.6980

[16] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," arXiv:1803.01271, 2018. https://arxiv.org/abs/1803.01271

[17] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014. https://jmlr.org/papers/v15/srivastava14a.html

[18] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html

[19] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," arXiv:1912.01703, 2019. https://arxiv.org/abs/1912.01703

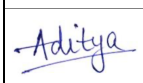
[20] C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," arXiv:1906.08172, 2019. https://arxiv.org/abs/1906.08172

INDIVIDUAL CONTRIBUTION STATEMENT

Project: CalmGesture

The statements below follow the model responsibilities assigned in the project synopsis. Each statement identifies the corresponding technical scope within the shared CalmGesture pipeline.

INDIVIDUAL CONTRIBUTIONS AND SIGNATURES

Member Name	Reg. No.	Model / Task Owned	Key Contributions	Signature
Sangini Singh	230911020	MLP	My contribution area is the MLP baseline: temporal mean and standard-deviation features, training-only standardization, the 64/32-unit classifier, and interpretation of its accuracy and macro-F1. This responsibility also covers explaining the limitations of discarding temporal order in the model comparison.	
Tanvi Gupta	230911122	CNN	My contribution area is the spatial CNN: landmark summary inputs, the NumPy convolutional architecture, training configuration and interpretation of its results. This responsibility includes documenting the gradient-scaling correction and distinguishing the implemented model from the proposed framework port.	
Aditya Nigam	230911366	1D-CNN	My contribution area is the temporal 1D-CNN: sequence inputs, temporal convolutions, pooling, dropout and validation-based checkpoint selection. This responsibility covers interpretation of the selected model's class errors and its role in the local gesture-to-sound integration.	
Shubh Rastogi	230953258	GRU	My contribution area is the GRU model: recurrent sequence processing, the hidden-state representation, training settings and comparison with convolutional alternatives. This responsibility includes discussing temporal dependencies, generalization limits and the need for further continuous-input evaluation.	